

Component-1 Design Note — lexical & semantic retrieval on EB-NeRD and MIND

1. What was built

A config-driven pipeline from raw archives to metric reports in one command — `make data` for Q1’s feature store, `make all` for everything — across **EB-NeRD demo** (49k impressions / 1,590 users), **EB-NeRD small** (478k / 15,143) and **MIND-small** (230k / 50,000). Both datasets normalise into one schema (`impressions_{train,val,test}.parquet`: id, user, timestamp, candidates[], clicked[], history[], recency_seconds; `articles.parquet`: id, title, abstract, body, category, entities[], published_time), so every downstream stage has one code path, not a Danish branch and an English one. `entities` normalises two source shapes (EB-NeRD’s `ner_clusters`, MIND’s per-field JSON) into one `List(String)` (74%/87% non-empty); nothing scores on entities yet, the feature store is a contract.

Three scorers run over that schema, **bm25**, **emb**, **fused**, on two tracks: **re-rank** (score the pool the log actually showed; leaderboard-comparable) and **retrieval** (ignore the pool, pull the top-200 from the whole corpus, the “candidate generation to a few hundred” the spec asks for). The query for both is the user’s last 30 clicked titles; the harness’s own metrics are in §7.

2. Temporal split

Both datasets ship their own train/test boundary, so I adopted it as `test_start` rather than inventing one, and carved `val` out of the earlier block by time (EB-NeRD small train/val/test: 168,522/64,365/244,647; MIND-small: 95,071/61,894/73,152 impressions). EB-NeRD’s day boundary is **07:00**, not midnight, a detail that only shows up in the data.

EB-NeRD ships a separate `history.parquet` per block whose window ends exactly where its impressions begin, so the behaviour-window boundary is given rather than guessed (NOTES.md, §splits, for the alternative considered and rejected).

Enforcement, not discipline. `split.py` asserts on every build and fails rather than warns: strict `train < val < test`; per-item history-precedes-impression; `clicked` is a subset of `candidates`; every candidate resolves; no empty ids. Where a check is impossible it says so — MIND’s history has no per-item timestamps, so the harness prints that the guarantee is structural rather than verified, instead of passing vacuously.

`tests/test_leakage.py` (make `test`, 19 tests, ~20s) re-checks the same properties from *outside*, reading only `data/processed/`: the build asserts what it meant to write, the tests assert what is on disk, plus two checks that exist only there (`datasets.yaml` cutoffs still match the artifacts; no module outside `eval/run.py` mentions a serving-unavailable column). Verified non-vacuous by injecting a future click and watching it fire.

3. Lexical axis

bm25s over title+abstract with a per-dataset Snowball stemmer (**rank_bm25** scores one query at a time in pure Python, unusable at 65k documents; Pyserini/Lucene scales further but drags in a JVM).

Stemming is not free, and not what I assumed. The brief specifies no preprocessing, so this was my choice. On `val`, `stem - none` is **+0.0019 [+0.0007, +0.0029] AUC on MIND, zero on EB-NeRD, and significantly negative on recall@200 for both** (-0.0020, -0.0023): it trades precision for variant-matching — worth it ordering ~8 candidates, harmful picking 200 from 20,738. Kept as right for the headline metric, but a track-dependent trade. Danish compound splitting from corpus vocabulary fired on 15% of tokens and made both **worse**: compounding needs a lexicon, not statistics.

The finding that mattered. EB-NeRD first scored AUC **0.5035**, random. Rather than report that, I checked whether it was my bug: an **oracle test** (query = the clicked article’s own title) ranked it #1 in 300/300 impressions at AUC 0.9990, and a history-length sweep was flat, ruling out misalignment and query dilution. MIND worked at 0.58; that asymmetry located the cause: **bm25s ships no Danish stopword list**, so thirty concatenated Danish titles were mostly function words. The Snowball list moved EB-NeRD to **0.5232**. A missing stopword list is silent, it degrades without erroring.

4. Semantic axis

MIND encodes title+abstract with **all-MiniLM-L6-v2 (11m16s)**, cached once, 65,238-article union across its two files). User vectors are the mean of last-30 history vectors, L2-normalised; scoring is cosine with FAISS `IndexFlatIP` (exact at this scale; IVF/PQ deferred to 10× analysis).

The encoder is a measurement, not a preference. Q3 allows the provided article vectors *or* your own from BERT/XLM-RoBERTa, so I ran the identical pipeline on each candidate: same splits, same BM25, same fusion, only the article vectors differ. Selection is on `val`, since selecting on test tunes a choice on the number being reported. Semantic-only AUC, `ebnerd_small val`, against BM25’s 0.5205:

encoder	arm	dim	val AUC	
Ekstra_Bladet_contrastive_vector (ships)	provided	768	0.5506 [0.5480, 0.5531]	beats BM25
paraphrase-multilingual-mpnet-base-v2 (XLM-R)	word2vec	768	0.5309 [0.5282, 0.5336]	beats BM25
Ekstra_Bladet_word2vec	provided	300	0.5098 [0.5073, 0.5124]	loses
google_bert_base_multilingual_cased	provided	768	0.4857 [0.4832, 0.4880]	below random

On shipping contrastive_vector: it is an official EB-NeRD artifact from the same `artifacts/` folder as the other two, so it sits in Q3’s “provided article embeddings” arm, word2vec and mBERT are that arm’s named examples, not its full extent. Strongest EB-NeRD encoder measured, by +0.0197 val AUC over my own XLM-R, gap well outside both intervals.

One property predicts all four: **was the model trained so cosine distance means topical similarity?** Contrastive training optimises exactly that and wins; XLM-R paraphrase fine-tuning approximates it and comes second; word2vec (predict neighbouring words) does not; raw mBERT (fill in blanks) emphatically does not, its vectors sitting in a narrow cone where everything looks alike. Size predicts nothing: the 300-d model beats one 768-d model. Fusion detects the mBERT failure unprompted, tuning alpha to 0.00 on demo, discarding the semantic axis outright.

MIND kept MiniLM, and that is the sharper lesson. Against the same XLM-R, MiniLM wins on **val** (0.6338 vs 0.6256) and *loses* on **test** (0.6339 vs 0.6364), a real reversal, near-disjoint intervals. The rule was fixed before looking, so MiniLM stays; “take the better test number” would have flipped it, costing 0.0025 test AUC to obey, the entire point of having a rule. An English specialist beats a 50-language generalist on English, while that generalist beats a 2013 word-vector model on Danish.

5. Candidate generation: recall@K, and a split-dependent winner

Recall@K over the **full catalogue**, not the pool the log showed, is what measures candidate generation: the share of an impression’s clicked articles found in the top K. Cold-start impressions retrieve nothing and score 0 rather than being excluded, since a retriever that cannot serve a user has failed for that user.

recall@200 (small; demo tracked it within noise, §8)	bm25	emb	emb – bm25
EB-NeRD small · val	0.0214	0.0198	−0.0016 [−0.0030, −0.0002]
EB-NeRD small · test	0.0247	0.0277	+0.0030 [+0.0022, +0.0039]
MIND-small · test	0.0220	0.0239	+0.0019 [+0.0008, +0.0031]

On EB-NeRD the retrieval winner flips between val and test, both directions significant: BM25 wins val, embeddings win test, same code, same K, a sign change with disjoint intervals, not noise. MIND shows no such flip; embeddings win there at every K. §6 finds the identical pattern in the fusion weight, pointing at the dataset rather than either method: val is a 2-day window against test’s 7, each with its own history window, so the lexical/semantic relationship genuinely drifts across the boundary, two weeks is a different news distribution.

Re-ranking and retrieving stay different jobs, but by margin now, not direction. The same contrastive vectors beat BM25 by +0.0289 AUC ordering the given pool and by only +0.0030 recall@200 searching all 20,738 articles, a 10x gap for the same encoder on the same split, evidence for different signals per stage rather than one universal winner. Absolute recall is low everywhere (2–5%); §8 explains why, a property of the corpus, not either retriever.

6. Fusion, an addition the brief did not ask for

Q3 asks only to *compare* lexical and semantic; this third system is mine. BM25 scores and cosines are on different scales, and BM25’s moves with query length, so both are z-normalised **within each candidate pool** and mixed $\alpha*\text{emb} + (1-\alpha)*\text{bm25}$, alpha tuned on **val**, applied unchanged to test.

AUC (small; demo tracked it within noise, §8)	bm25	emb	fused	alpha	fused — emb
EB-NeRD small val	0.5205	0.5506	0.5528	0.70	+0.0022 [+0.0010, +0.0033]
EB-NeRD small test	0.5107	0.5397	0.5380	0.70	−0.0017 [−0.0023, −0.0011]
MIND-small test	0.5685	0.6369	0.6381	0.75	+0.0012 [+0.0005, +0.0018]

On EB-NeRD fusion wins on val and significantly loses on test, on both bundles: the alpha chosen on val does not generalise across the boundary. This is the same reversal §5 finds in retrieval, reached through a different measurement, a property of EB-NeRD, not an artefact of either method.

Consequently **the reported EB-NeRD system is embeddings alone**, and only MIND ships fused. Fusion still earns its place: it is the mechanism that *detected* the instability, and on MIND it is a real if small win. Judged on val alone, the tempting shortcut, I would have shipped fusion on all three and been wrong on two.

history_len was shared across datasets and shouldn’t have been. A sweep on MIND val found the constant (30) truncating a real tail: MIND’s median history is 20, not EB-NeRD’s 258, so 30 discarded signal rather than noise. AUC rose significantly to 100 clicks (bm25 +0.0039, emb +0.0018) then plateaued exactly there. EB-NeRD’s own sweep (§3) was flat over the same range, so it stays at 30; **history_len** is now per-dataset in config. This table reflects the corrected MIND value.

7. Evaluation harness

Metrics are computed **per impression**, which makes everything else possible: slicing is a mask over that vector, bootstrapping resamples it over impressions, the approximately-independent unit (resampling candidates within one would understate variance). Comparisons use a **paired** bootstrap on shared resamples, since two systems on the same impressions have correlated errors, an unpaired comparison of independent intervals is far too conservative.

Cold-start needed a defensible definition. “Empty history” fails on EB-NeRD, which has *zero* such users (median history 258), it would select an empty slice and silently report nothing. The harness uses a per-dataset bottom-decile history-length threshold plus the true zero-history slice where one exists; on MIND that slice scores 0.5125 identically across all three systems, correctly, no history means no query, so no ranking is invented.

Beyond-accuracy over the top-10: category intra-list diversity, novelty as self-information against *train* click shares, coverage, sobering at **5–21%** of the catalogue ever surfaced.

8. Observations

Content signals are weak here, and that is the result, not a failure. MIND reaches 0.6381, within reach of published neural baselines for MIND-small (~0.65–0.67) from an entirely unsupervised system. EB-NeRD tops out at 0.5397: its pools are much smaller (median 8–9 vs MIND’s 23–26) and drawn from a tight recency window, so every candidate is already plausible, there is little for content to separate.

Simple behavioural features are not the shortcut they look like. Train-click popularity scores *below* random on EB-NeRD (0.459), the pool is already popularity-filtered.

Demo was representative. Every system landed within 0.002 of its demo value at 10× the users; the extra data bought precision, CIs tightened from ±0.0040 to ±0.0013, deciding fusion.

The retrieval track is weak regardless of scorer (recall@200 of 2–5%): EB-NeRD’s corpus spans 1998–2023 while impressions are one week in May 2023, and the publication filter removes only *future* articles, not stale ones. A recency window on the retrievable corpus is the obvious C2 lever.

9. Anti-gaming and serving availability

total_inviews/total_pageviews/total_read_time are lifetime aggregates over the whole collection period, embedding the future, and cannot be computed at serving time. Carried into the corpus **only** so the harness can quantify them; no

shipped scorer reads them. Test AUC without/with popularity: EB-NeRD demo 0.5300 -> 0.5728, EB-NeRD small 0.5291 -> 0.5731 (+0.0440 [+0.0431, +0.0449]).

One serving-unavailable feature is worth +0.044, more than twice the entire semantic axis (+0.0198, §6), the largest legitimate win in this project, and the biggest effect anywhere in the report; any comparison that quietly includes it is not measuring the same system.

Also enforced, not trusted: post-click engagement columns are dropped at the split, and retrieval filters `published_time <= impression_time` (867 EB-NeRD articles postdate the test window).

10. Where it breaks at 10×, measured, not projected

The leaderboard submission made this section measured rather than hypothetical: Codabench scores `ebnerd_testset` at **13.5M impressions, 205,925,868 candidate pairs, 28× `ebnerd_small`**.

Memory is the wall, and it is one specific idiom. Three separate times the break was `.to_list()` pulling Arrow into boxed Python objects: `np.vstack(col.to_list())` peaks at **5.8 GB to build a 385 MB array**, the same idiom OOM-killed the harness at 10 GB RSS adding `recall@K`. `explode().to_numpy()` plus Polars set intersections fixed all three, byte-identically.

Streaming works and costs little. `pipeline/submit.py` scores in 200k-impression slices: peak RSS stays **flat against dataset size, not linear**, in **13m30s** on the current contrastive-vector build (33m28s at the same peak RSS under the earlier XLM-R build); MIND-large’s 2.4M took 6m05s at 1.3 GB. Slice pushdown reaches the parquet row groups, so an offset-13M slice costs the same as one at 0; without it streaming would be $O(n^2)$. MIND needed a one-off TSV→parquet conversion first, CSV has no such pushdown and OOM-killed the first attempt reading it whole (3.07 GB to parse).

BM25 is the expensive piece, and I first overstated how expensive. `get_scores` is dense over the whole corpus per distinct query, ~2M MIND-large histories × 120,961 articles, which I originally called impractical on the ~ 10^{11} operation count alone. Timing it says otherwise: 1.6 ms/query over 65,238 articles, extrapolating to **~1.6 h** at MIND-large’s corpus and query count. Slow, roughly 7× the embedding submission’s own runtime, but not a wall. Operation counts are not wall-clock when the kernel is sparse and vectorised, and I should have measured before asserting. The direction of the fix is unchanged: a dense scorer per query is the wrong shape at this size, and an inverted index with early termination (WAND) is what removes the cost rather than paying it. Projected at `ebnerd_large` (600M): FAISS flat is $O(N \cdot d)$ /query and would need IVF-PQ with `nlist ~ sqrt(N)`.

11. Leaderboard submissions

Both competitions score held-out sets far larger than the splits above (`ebnerd_testset`, 13.5M impressions; MIND-large test, 2.4M), streamed by `pipeline/submit.py` (§10) with unchanged scoring semantics. **Both entries are embeddings-only, not fused.** On EB-NeRD that *is* the reported system (§6). On MIND it costs a known 0.0012 AUC, and the reason is cost rather than impossibility: adding BM25 would take ~1.6 h on top of the current 5 min run (§10), and `submit.py` would need a BM25 path it does not have. A deliberate trade, not a blocked one, and reported as a different system rather than letting `fused`’s stronger offline number stand in for what was uploaded.

	submission	AUC	MRR	nDCG@5	nDCG@10
EB-NeRD (RecSys 2024, comp. 2469)	893893	0.5336	0.3412	0.3779	0.4576
MIND (comp. 13967)	892088	0.6460	0.3160	0.3405	0.3963

Each score must be read against the offline number for the system that produced it, not whatever the harness reports today, since both moved since these were built (EB-NeRD’s encoder, MIND’s `history_len`, §6). 893893 postdates the `contrastive_vector` switch, so its baseline is the current `emb` row (0.5397/0.3492/0.3831/0.4627): the leaderboard came in **below offline on all four metrics** (−0.0061 AUC), the expected direction, a large independent population regressing toward the mean rather than repeating one split’s luck. 892088 predates the `history_len` fix, so its baseline is the *old* `history_len=30` row (AUC 0.6339, MRR 0.3486, nDCG@5 0.3315, nDCG@10 0.3911): against that, leaderboard beat offline on three of four (AUC +0.0121, nDCG@5, nDCG@10), losing only on MRR (0.316 vs 0.349); likely `eval/metrics.py`’s first-clicked-article MRR versus an average-over-all-clicks implementation. Neither entry reflects the current pipeline; rebuilding both is the obvious next step, not done here, EB-NeRD is rate-limited to five submissions a day. Validated offline before upload: all 13,536,710 lines as rank permutations of the correct length in test-file row order, plus 40 per dataset re-ranked in float64 through an independent path. **Screenshots:** *(attach the two Codabench rows)*